

Class 6: R functions

Mario Gutierrez PID: A69043009

Table of contents

Our first (silly) function	1
A second function	2
A protein generating function	3

All functions in R have at least 3 things:

- A **name**, we pick this and use it to call our function.
- Input **arguments** (there can be multiple)
- The **body** lines of R coda that do the work.

Our first (silly) function

Write a function to add some numbers

```
add <- function(x, y=1) {  
  x + y  
}
```

Now we can call this function:

```
add(10,100)
```

```
[1] 110
```

```
add(c(10,10), 100)
```

```
[1] 110 110
```

A second function

Write a function to generate random nucleotide sequences of a user specific length:

The `sample()` function can be helpful here.

```
v <- sample(c("A", "C", "G", "T"), size = 50, replace = TRUE)
v
```

```
[1] "G" "G" "A" "T" "G" "C" "T" "C" "G" "A" "C" "T" "T" "T" "C" "A" "A" "C" "A"
[20] "G" "T" "T" "C" "T" "A" "A" "C" "C" "C" "G" "G" "A" "T" "A" "C" "A" "A" "A"
[39] "G" "T" "T" "A" "T" "T" "T" "T" "A" "C" "G" "C"
```

I want the a 1 element long character vector that looks like this “CACAGC” not “C” “A” “C” “A” “G” “C”.

```
paste(v, collapse = "")
```

```
[1] "GGATGCTCGACTTTCAACAGTTCTAACCCGGATACAAAGTTATTTTACGC"
```

Turn this into my first function:

```
generate_dna = function(size=50) {
  v = sample(c("A", "C", "G", "T"), size=size, replace = T)
  paste(v, collapse = "")
}
```

Test it:

```
generate_dna(60)
```

```
[1] "TGCCGAGCACATTTCCTTGAGAGAGGACTTCGTTTGGGAGGTGCGCTGGACACGCCCATCA"
```

```
fasta <- TRUE
if(fasta){
  cat("HELLO You!")
} else{
  cat("No you dont!")
}
```

HELLO You!

Add the ability to return a multi-element vector or a single element fasta like a vector.

```
generate_fasta = function(size=50, fasta=TRUE) {  
  v = sample(c("A", "C", "G", "T"), size=size, replace = T)  
  s <- paste(v, collapse = "")  
  if(fasta){  
    return(s)  
  } else{  
    return(v)  
  }  
}
```

```
generate_fasta(10)
```

```
[1] "CCCCGGTGAG"
```

A protein generating function

```
generate_protein = function(size=50, fasta=TRUE) {  
  v = sample(c("A", "R", "N", "D", "C", "Q", "E", "G", "H", "I", "L", "K", "M", "F", "P", "S"), size=size, replace = T)  
  s <- paste(v, collapse = "")  
  if(fasta){  
    return(s)  
  } else{  
    return(v)  
  }  
}
```

```
generate_protein(6)
```

```
[1] "LFMACL"
```

Use our new `generate_protein()` function to make random protein sequences of length 6 to 12 (i.e. one length 6, one length 7, etc. up to 12).

One way to do this is “brute force”:

```
generate_protein(6)
```

```
[1] "CVIDLR"
```

```
generate_protein(7)
```

```
[1] "TTQSAFL"
```

```
generate_protein(8)
```

```
[1] "KFESQWKA"
```

```
generate_protein(9)
```

```
[1] "FSEDTEFRG"
```

```
generate_protein(10)
```

```
[1] "RRWVQQGKKQ"
```

```
generate_protein(11)
```

```
[1] "GCCKSYLNKEH"
```

```
generate_protein(12)
```

```
[1] "TAFSDYLVHNGV"
```

A second way is to use a `for()` loop:

```
lengths <- 6:12  
lengths
```

```
[1] 6 7 8 9 10 11 12
```

```
for(i in lengths){
  cat(">", i, "\n", sep = "")
  aa <- generate_protein(i)
  cat(aa)
  cat("\n")
}
```

```
>6
PEVMLS
>7
DQAPFSC
>8
RYTLCGIR
>9
PDHNYWYCH
>10
QPVFCNPTWD
>11
FTCAACECPWV
>12
CNLFYINEDKKK
```

```
paste(c("barry", "monika"), sep = "")
```

```
[1] "barry" "monika"
```

A third, and better, way to solve this is to use the `apply()` family of functions, specifically the `sapply()` function in this case.

```
sapply(6:12, generate_protein)
```

```
[1] "FAWPTA"      "TQHQLPL"      "VPSRDGWP"      "FTVYMATDW"      "ACGDITFAAC"
[6] "PNAHANTGTNR" "EYQYYGVDCTFW"
```